

POS SaaS Backend Documentation

Golang + PostgreSQL + Docker + Goose + JWT + Role Access

Setup Guide, API Testing Guide, and Backend Flow

Module	Status
Auth JWT + Role Access	Ready
Store, Branch, Category, Product	Ready
Product Image Upload	Local Storage Ready
Transaction, Void, Receipt	Ready
Daily Sales Report + Dashboard	Ready

Dokumen ini dibuat sebagai panduan teknis untuk menjalankan, memahami, dan menguji backend POS SaaS. Cocok untuk disertakan bersama source code ebook atau produk digital.

Daftar Isi

- 1. Project Overview
- 2. Fitur Utama
- 3. Struktur Project
- 4. Requirement dan Setup
- 5. Docker PostgreSQL dan Goose Migration
- 6. Menjalankan Backend
- 7. Default User dan Login
- 8. Endpoint Overview
- 9. API Testing Guide
- 10. Role Based Access Testing
- 11. Product Image Upload
- 12. Transaction, Void, Receipt
- 13. Report, Dashboard, RawBT Note
- 14. Error Umum
- 15. Pengembangan Selanjutnya

1. Project Overview

POS SaaS Backend adalah project backend POS berbasis Golang yang dibangun untuk kebutuhan pembelajaran real project. Project ini bukan hanya CRUD sederhana, tetapi sudah memiliki auth, role access, transaksi, stok, laporan harian, dashboard, dan receipt API.

Stack utama yang digunakan:

- Golang
- Gin Web Framework
- PostgreSQL
- Docker Compose
- Goose Migration
- JWT Authentication
- Local Image Upload

2. Fitur Utama

Auth

- Login menggunakan email dan password
- Generate JWT access token
- Protected route menggunakan Bearer token
- Endpoint authenticated user

Role Access

- superadmin dapat mengakses semua store dan branch
- admin hanya dapat mengakses data store miliknya
- cashier hanya dapat mengakses data branch miliknya

Master Data

- Store CRUD
- Branch CRUD
- Category CRUD
- Product CRUD
- Product image local upload
- Product filter, search, dan pagination

POS Transaction

- Create transaction
- List dan detail transaction
- Stock product otomatis berkurang saat transaksi
- Void transaction
- Stock product otomatis kembali saat transaksi di-void
- Receipt API untuk struk

Report dan Dashboard

- Daily sales report
- Payment breakdown
- Top products
- Dashboard summary
- Sales per hour
- Low stock products
- Recent transactions

3. Struktur Project

```
pos-saas-backend/  
├── cmd/  
│   ├── api/  
│   └── main.go  
├── docs/  
│   └── API_TESTING.md  
├── internal/  
│   ├── auth/  
│   ├── config/  
│   ├── database/  
│   ├── handlers/  
│   ├── helpers/  
│   ├── middlewares/  
│   ├── models/  
│   ├── repositories/  
│   ├── requests/  
│   ├── responses/  
│   ├── routes/  
│   └── services/  
├── migrations/  
├── uploads/  
├── products/  
├── docker-compose.yml  
├── go.mod  
├── go.sum  
├── .env.example  
└── README.md
```

4. Requirement dan Setup

Pastikan sudah install: Go, Docker, Docker Compose, Goose, curl, dan jq.

```
go version  
docker --version  
docker compose version  
goose -version
```

Install Goose jika belum ada

```
go install github.com/pressly/goose/v3/cmd/goose@latest
```

Clone dan setup env

```
git clone URL_REPOSITORY  
cd pos-saas-backend  
cp .env.example .env
```

Contoh .env local

```
APP_NAME=POS SaaS Backend  
APP_ENV=local  
APP_PORT=8080  
APP_DEBUG=true  
APP_BASE_URL=http://localhost:8080
```

```
DB_HOST=localhost
DB_PORT=5433
DB_DATABASE=pos_saas_db
DB_USERNAME=pos_user
DB_PASSWORD=pos_password
DB_SSLMODE=disable
```

```
JWT_SECRET=pos_saas_local_secret_change_me
JWT_ACCESS_TOKEN_EXPIRES_MINUTES=60
```

5. Docker PostgreSQL dan Goose Migration

Jalankan PostgreSQL

```
docker compose up -d postgres
docker ps
```

Masuk PostgreSQL

```
docker exec -it pos_saas_postgres psql -U pos_user -d pos_saas_db
```

Run migration

```
goose -dir migrations postgres "host=localhost port=5433 user=pos_user password=pos_password dbname=pos_saas_db
sslmode=disable" up
```

Cek status migration

```
goose -dir migrations postgres "host=localhost port=5433 user=pos_user password=pos_password dbname=pos_saas_db
sslmode=disable" status
```

Rollback satu migration

```
goose -dir migrations postgres "host=localhost port=5433 user=pos_user password=pos_password dbname=pos_saas_db
sslmode=disable" down
```

6. Menjalankan Backend

```
go mod tidy
go run ./cmd/api
```

Jika berhasil, server berjalan di <http://localhost:8080>.

Health Check

```
curl http://localhost:8080/health
curl http://localhost:8080/api/health
curl http://localhost:8080/api/db-ping
```

7. Default User dan Login

Role	Email	Password
Superadmin	admin@possaas.test	password
Admin Main Store	admin.store@possaas.test	password
Cashier Main Branch	cashier.branch@possaas.test	password
Admin Second Store	admin.second@possaas.test	password
Cashier Second Branch	cashier.second@possaas.test	password

Login superadmin

```
curl -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin@possaas.test","password":"password"}'
```

Simpan token

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin@possaas.test","password":"password"}' | jq -r '.data.access_token')

echo $TOKEN
```

Auth Me

```
curl http://localhost:8080/api/auth/me -H "Authorization: Bearer $TOKEN"
```

8. Endpoint Overview

Group	Method	Endpoint	Keterangan
Auth	POST	/api/auth/login	Login user
Auth	GET	/api/auth/me	Ambil user login
Store	GET	/api/stores	List store
Store	POST	/api/stores	Create store
Store	GET	/api/stores/:id	Detail store
Store	PUT	/api/stores/:id	Update store
Store	DELETE	/api/stores/:id	Delete store
Branch	GET	/api/branches	List branch
Branch	POST	/api/branches	Create branch
Branch	GET	/api/branches/:id	Detail branch
Branch	PUT	/api/branches/:id	Update branch
Branch	DELETE	/api/branches/:id	Delete branch
Category	GET	/api/categories	List category
Category	POST	/api/categories	Create category
Category	GET	/api/categories/:id	Detail category
Category	PUT	/api/categories/:id	Update category
Category	DELETE	/api/categories/:id	Delete category
Product	GET	/api/products	List product
Product	POST	/api/products	Create product
Product	GET	/api/products/:id	Detail product
Product	PUT	/api/products/:id	Update product
Product	DELETE	/api/products/:id	Delete product
Transaction	GET	/api/transactions	List transaction
Transaction	POST	/api/transactions	Create transaction
Transaction	GET	/api/transactions/:id	Detail transaction
Transaction	POST	/api/transactions/:id/void	Void transaction
Transaction	GET	/api/transactions/:id/receipt	Receipt transaction
Report	GET	/api/reports/daily-sales	Daily sales report
Dashboard	GET	/api/dashboard	Dashboard summary

9. API Testing Guide

Bagian ini berisi curl command utama untuk test flow backend.

Login semua role

```
SUPERADMIN_TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json"
-d '{"email":"admin@possaas.test","password":"password"}' | jq -r '.data.access_token')
```

```
ADMIN_TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin.store@possaas.test","password":"password"}' | jq -r '.data.access_token')
```

```
CASHIER_TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d
'{"email":"cashier.branch@possaas.test","password":"password"}' | jq -r '.data.access_token')
```

10. Role Based Access Testing

Superadmin lihat semua store

```
curl http://localhost:8080/api/stores -H "Authorization: Bearer $SUPERADMIN_TOKEN"
```

Admin Main hanya lihat store miliknya

```
curl http://localhost:8080/api/stores -H "Authorization: Bearer $ADMIN_TOKEN"
```

Cashier tidak boleh akses stores

```
curl http://localhost:8080/api/stores -H "Authorization: Bearer $CASHIER_TOKEN"
```

Admin paksa akses store lain

```
curl "http://localhost:8080/api/products?store_id=2&page=1&limit=10" -H "Authorization: Bearer $ADMIN_TOKEN"
```

Cashier paksa akses branch lain

```
curl "http://localhost:8080/api/products?branch_id=2&page=1&limit=10" -H "Authorization: Bearer
$CASHIER_TOKEN"
```

11. Product Image Upload

Product image menggunakan multipart/form-data. Field file bernama image. Format yang diperbolehkan: jpg, jpeg, png, webp. Maksimal ukuran file 2MB. File disimpan ke uploads/products/ dan dapat diakses dari APP_BASE_URL.

Create product tanpa image

```
curl -X POST http://localhost:8080/api/products -H "Authorization: Bearer $TOKEN" -F "store_id=1" -F
"branch_id=1" -F "category_id=1" -F "name=Nasi Goreng" -F "sku=NSG-001" -F "barcode=899000000001" -F
"description=Nasi goreng spesial" -F "cost_price=15000" -F "selling_price=25000" -F "stock=20" -F
"min_stock=5" -F "is_active=true"
```

Create product dengan image

```
curl -X POST http://localhost:8080/api/products -H "Authorization: Bearer $TOKEN" -F "store_id=1" -F
"branch_id=1" -F "category_id=1" -F "name=Mie Goreng" -F "sku=MIE-001" -F "barcode=899000000002" -F
"description=Mie goreng spesial" -F "cost_price=12000" -F "selling_price=22000" -F "stock=15" -F
"min_stock=5" -F "is_active=true" -F "image=@$HOME/Pictures/product.jpg"
```

List product dengan filter

```
curl
"http://localhost:8080/api/products?store_id=1&branch_id=1&category_id=1&is_active=true&search=mie&page=1&limit=10"
-H "Authorization: Bearer $TOKEN"
```

12. Transaction, Void, Receipt

Create transaction cash

```
curl -X POST http://localhost:8080/api/transactions -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{
  "store_id": 1,
  "branch_id": 1,
  "customer_name": "Customer Walk In",
  "payment_method": "cash",
  "cash_amount": 50000,
  "transfer_amount": 0,
  "notes": "Test transaksi Mie Goreng",
  "items": [
    {
      "product_id": 2,
      "qty": 1,
      "discount": 0
    }
  ]
}'
```

Detail transaction

```
curl http://localhost:8080/api/transactions/1 -H "Authorization: Bearer $TOKEN"
```

Receipt transaction

```
curl http://localhost:8080/api/transactions/1/receipt -H "Authorization: Bearer $TOKEN"
```

Void transaction

```
curl -X POST http://localhost:8080/api/transactions/1/void -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{
  "reason": "Salah input transaksi"
}'
```

13. Report, Dashboard, RawBT Note

Daily sales report

```
curl "http://localhost:8080/api/reports/daily-sales?date=2026-05-19&branch_id=1" -H "Authorization: Bearer $TOKEN"
```

Dashboard

```
curl "http://localhost:8080/api/dashboard?date=2026-05-19&branch_id=1" -H "Authorization: Bearer $TOKEN"
```

RawBT Note

RawBT berada di sisi frontend. Backend hanya menyediakan data receipt lewat endpoint GET `/api/transactions/:id/receipt`. Frontend React mengambil data tersebut, membuat tampilan struk, generate teks struk, lalu membuat link `rawbt://print` yang dibuka dari browser Android. RawBT app di device kasir yang menangani proses print ke printer thermal.

14. Error Umum

Error	Penyebab	Solusi
Authorization header is required	Tidak mengirim Authorization header	Kirim -H Authorization: Bearer \$TOKEN
Authorization header must use Bearer token	Token kosong atau format salah	Cek echo \$TOKEN dan login ulang
invalid email or password	Email/password salah	Gunakan user default dan password: password
duplicate key value violates unique constraint	Kode unik sudah ada	Ganti store code, branch code, category code, atau product sku

Error	Penyebab	Solusi
goose command not found	Goose belum terinstall atau PATH belum benar	Install goose dan cek PATH
insufficient stock	Qty transaksi lebih besar dari stock	Kurangi qty atau adjust stock

15. Pengembangan Selanjutnya

- Stock movement history
- Stock adjustment
- User management
- Refresh token
- Upload image ke S3 atau GCP Cloud Storage
- Export report Excel/PDF
- API rate limiting
- Unit test
- React frontend
- RawBT print integration dari frontend

16. Urutan Test Paling Aman

1. Health check
2. Login superadmin
3. Auth me
4. Create category
5. Create product
6. List product
7. Create transaction
8. Detail transaction
9. Receipt transaction
10. Daily sales report
11. Dashboard
12. Void transaction
13. Cek stock product balik
14. Cek daily sales report setelah void
15. Cek dashboard setelah void